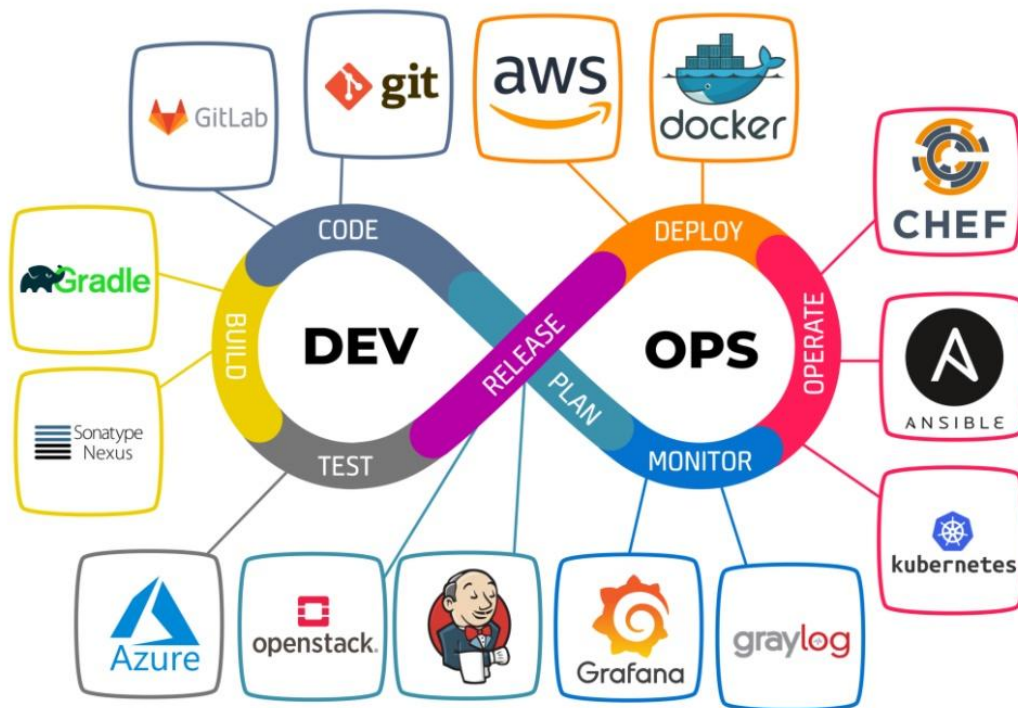


Integration of Cloud Computing and DevOps :

A study of famous DevOps tool – Docker

Authors: Krishna Agarwal, Kirti Agarwal



Chapter's Outline

Introduction

- Overview of DevOps
- Need of DevOps
- Evolution of DevOps

Section 1: Cloud DevOps – The Future of Development

1.1 Understanding Cloud DevOps

- Definition and concept of Cloud DevOps
- How do DevOps and Cloud interact?

1.2 Role of DevOps in Cloud Computing

- Infrastructure Automation
- CI/CD
- Cloud Resource Optimization
- Security and Compliance
- Monitoring & Logging
- Collaborative & Agile Development

1.3 Benefits of DevOps in Cloud Computing

- Faster Software Delivery
- Cheap Cost
- Better Corporation
- Enhanced Security
- Scalability

Section 2: Continuous Integration and Continuous Development (CI/CD)

2.1 Introduction to CI/CD

- Definition of CI/CD
- Introduction to CI/CD Pipeline
- Components of CI/CD

2.2 Advantages of CI/CD

- Quicker Releases
- Early Bug Detection

- Scalability
- Greater Corporation
- Improved Safety

2.3 Challenges in CI/CD

- Difficulty in Setup
- Cost of Infrastructure
- Security Threats
- Steep Learning Curve

Section 3: Containerization

3.1 Introduction to Containerization

- Overview of Containerization
- How does Containerization works?
- Why is Containerization favored?
- Essential Components of Containerization

3.2 Containerization vs Virtual Machines

- Engineering
- Asset Utilization
- Startup Time
- Versatility
- Transportability
- Segregation
- Execution
- Utilize Cases

Section 4: Docker Architecture

4.1 Introduction to Docker

- Overview of Docker

4.2 Key Components of Docker

- Docker Hub
- Docker Desktop
- Docker Engine
- Docker Daemon
- Docker Image

- Docker Container

Section 5: Advanced Docker Concepts

5.1 Introduction to dockerfile

- Overview of dockerfile

5.2 Introduction to dockerignore file

- Overview of dockerignore File

5.3 Docker Networking

- Overview of Docker Network

5.4 Docker Compose File

- Overview of Docker Compose File

5.5 Docker Volumes

- Overview of Docker Volume
- Working of Docker Volume

Section 6: Publishing Docker Images

6.1 Publishing Docker Image

- Steps to publish the image on Docker Hub

Section 7: Troubleshooting Commands

7.1 Overview of Troubleshooting Commands

- Logs of containers
- Interactive mode of Container

Section 8: Dockerizing our Application

8.1 Time to Dockerize your Application

- Steps required to Dockerize the Application

Section 9: Overview of Kubernetes

9.1 Introduction to Kubernetes

- Overview of Kubernetes
- Difference between Docker and Kubernetes

Introduction

Traditionally, the developers during old times used to write code and then pass it to the operations team, who will now deploy and maintain it. This process often leads to delays, miscommunication, and inefficiencies as developers have to wait for operations team to themselves deploy and configure everything, as the developers and operations team often work distantly which might lead to misunderstandings, testing and deployment happen late in the process, making errors and problems harder to fix, it also increases the lack of flexibility.

DevOps is a combination of Development (Dev) and Operations (Ops), designed to improve collaboration between software development and IT operations. DevOps is a combination of the development and operations of the term that refers to a collaborative or shared approach to the work done by an organization's application development and IT operations teams (4). DevOps is one of the methodologies used by IT professionals to finish IT projects that meet business needs.

The need for a new methodology was identified by Patrick Debois, a Belgian IT consultant, frustrated with slow and inefficient collaboration between developers and operations, so he started looking for ways or alternatives to improve the system in the year 2007-2008.

2009 can be considered as The birth of “DevOps”, At the Velocity Conference in 2009, John Allspaw and Paul Hammond (from Flickr) gave a talk titled "10+ Deploys a Day: Dev and Ops Cooperation at Flickr", demonstrating how development and operations teams could work together in harmony to deploy software multiple times a day, inspired Patrick Debois to organize the first "DevOps Days" conference in Belgium, where the term DevOps was officially introduced.

In the years 2010-2015 big companies like Netflix, Amazon, Google, and Facebook started adopting its practices to achieve faster and more reliable software releases. Presently it has become the new standard, adopted across industries, helping companies deliver software faster, safer, and more efficiently.

The primary purpose of using DevOps method is to use automation that would help in speeding up the repetitive tasks and reduce their downtime. Tasks can be performed automatically, and nobody will have to wait because they will be able to interact more closely and produce faster and more accurate results. Its main purpose is to have more work done quickly without exhausting the workers.

Section 1: Cloud DevOps – The Future of Development

1.1 Understanding Cloud DevOps

DevOps is a convergence of development (Dev) and operation (Ops) that depends on computerization, collaboration, and continuous delivery. When combined with cloud computing, DevOps simplify software development and deployment through scalable, flexible, and automated cloud based solutions.

1.1.1 How Do DevOps and Cloud Interact?

Cloud provides on-demand infrastructure: DevOps employs it for fast deployment.

Cloud services host computerized software (CI/CD pipelines) that integrate, test, and deploy automatically.

Scalability & Flexibility: Cloud facilitates DevOps teams to scale up or down on demand.

Security & Monitoring DevOps utilize cloud-based software and monitoring services.

1.2 Roles of DevOps in Cloud Computing

1. Infrastructure Automation

Utilizes Infrastructure as Code (IaC) tools like Terraform, Ansible, and AWS CloudFormation to automate infrastructure provisioning.

Decreases human interaction and human errors in cloud resource management.

2. Continuous Integration & Continuous Deployment (CI/CD)

Cloud DevOps enables automatic deployment, tests, and build through Jenkins, GitHub Actions, GitLab CI/CD, and CircleCI.

Facilitates timely releases and effective deployment of the software.

3. Cloud Resource Optimization

Cloud monitoring software (AWS CloudWatch, Azure Monitor, Prometheus) is used by DevOps engineers to reduce costs and resource utilization.

Supports auto-scaling of applications against real-time demand.

4. Security & Compliance

DevSecOps brings defense into cloud DevOps processes by using automated security scans.

Leverages cloud security products (AWS IAM, Azure Security Center, HashiCorp Vault) to offer conformity.

5. Monitoring & Logging

Use technologies like ELK Stack (Elasticsearch, Logstash, Kibana) ,Prometheus and Grafana for cloud application monitoring.

Helps with performance tuning and debugging.

6. Collaboration & Agile Development

Cloud DevOps enable remote collaboration using services like GitHub, GitLab, and Bitbucket. Supports fast development with quick feedback loops.

1.3 Benefits of DevOps in Cloud Computing

Faster Software Delivery: Automates deployment and test.

Cheap Cost: Maximizes cloud resource utilization.

Better Cooperation: Ops and development teams coexist harmoniously.

Enhanced Security: Automated protection scans and compliance enforcement.

Scalability: Cloud allows dynamic application scaling.

Section 2: Continuous Integration and Continuous Development (CI/CD)

2.1 Introduction to CI/CD

CI/CD is a DevOps practice in which software development, testing, and deployment are automated to promote quicker and more stable application delivery. It contributes to minimizing human effort, flagging bugs in the early stage, and enabling efficient delivery of high-quality software.

CI/CD has two central components:

Continuous incorporation (CI) - Automates code integration testing, and structure to capture problems early.

Continuous Deployment/Delivery (CD): Deploys automatically to production or staging environments.

2.1.1 CI/CD Pipeline

A CI/CD pipeline is a series of automated tasks to move code from development to production. It normally includes:

Code Commit: Developers commit code to a version control system (Git, GitHub, GitLab).

Build Stage: Code compilation and dependencies are installed.

Testing Stage: Automated tests (unit, integration, security tests) are executed.

Artifact Creation: A deployable package is produced (Docker image, binary files).

Deployment Stage: The app is deploy to staging or prod.

Monitoring & Feedback: Logs and performance are monitor to identify problems.

2.1.2 CI/CD Components

Version Control Systems: Git, GitHub, GitLab, Bitbucket

Build Automation Tools: Maven, Gradle

Testing Frameworks: JUnit, Selenium, Jest

CI/CD Tools: Jenkins, GitHub events, GitLab CI/CD, CircleCI, Travis CI

Containerization & Orchestration: Docker, Kubernetes

Infrastructure as Code (IaC): Terraform, Ansible

2.2 Advantages of CI/CD

Quicker Releases – Automates deployment.

Early Bug Detection – Continuous testing enhances code quality.

Scalability – accommodate large teams and multiple deployments.

Greater Cooperation – Teams cooperate effectively.

Improved Safety – programmed security scans for identify weaknesses.

2.3 Challenges in CI/CD

Difficulty in Setup – Needs setup of tools and pipelines.

Cost of Infrastructure – Cloud-based CI/CD services are costly.

Security Threats – Misconfigured pipelines may expose sensitive information.

Steep Learning Curve – Developers must have DevOps practice knowledge.

Section 3: Containerization

3.1 Introduction to Containerization

1. Presentation to Containerization

Consider containerization as bundling an application with everything it should run a code, conditions, and runtime into a slick, versatile bundle called a holder. Holders execute the same all over, be it on a dev's tablet, a cloud occasion, or in an on-prem information center. Consistency and consistency are given notwithstanding of where the app runs.

2. How Does Containerization Work?

Holders are comparative to light virtual machines (VMs), but they do not contain a full-fledged working framework for each program, but they do share the OS bit but isolated everything else.

Each holder has:

The application's code

Required libraries and conditions

Runtime environment in which to execute the application

This approach makes the applications versatile, versatile, and proficient, i.e., they can be executed anyplace without a compatibility issue.

3. Why Utilize Holders? (Why Containerization is favored)

Transportability: Say farewell to "works on my machine" issues containers work the same anyplace.

Lightweight: Not at all like VMs, holders share framework assets, making them much more effective.

Adaptability: Require extra handling control? Essentially turn up extra holders promptly.

Quick Sending: Holders begin in seconds, though VMs take minutes to boot.

Confinement: Each holder works autonomously, which offers more security and solidness.

4. Essential Components of Containerization

Holder Motor: The runtime that runs and oversees holders (e.g., containerd, Docker).

Holder Picture: A pre-configured template with everything it must run an application. It is comparable to a "diagram" for a holder.

Holder Registry: A store and administration store for pictures for holders (e.g., Docker Center, AWS ECR, Google Holder Registry).

Holder Organization: With more than one holder, coordination program such as Kubernetes or Docker Swarm takes over to computerize sending, scaling, and organizing.

Containerization revolutionized computer program improvement by speeding up apps, making them more secure, and basic to scale. Whether you're building a minor web application or a huge cloud based framework, holders guarantee everything works flawlessly, safely and efficiently, wherever you convey it.

3.2 Containerization vs Virtual Machines

Presentation

Both containerization and virtual machines (VMs) are innovations utilized to make confined situations for applications. Whereas they serve comparative purposes, they work in an unexpected way in terms of engineering, execution, and asset productivity.

Key Differences

Engineering

Holders share the have working framework bit and run in confined client spaces. This makes them lightweight and productive. In differentiate, VMs run on a hypervisor and have their possess working frameworks, making them more resource-intensive.

Asset Utilization

Holders utilize fewer system assets since they don't require a partitioned OS for each occurrence. VMs, on the other hand, expend more CPU, Smash, and storage because each VM incorporates a full OS at the side the application.

Startup Time

Holders begin in seconds since they as it were ought to dispatch the application and its conditions. VMs take minutes to boot since they must stack a total working framework.

Versatility

Holders are exceedingly adaptable, permitting different occurrences to be sent rapidly with negligible asset utilization. Scaling VMs requires running additional OS occasions, which

increments asset utilization and moderates down arrangement.

Transportability

Holders run reliably over distinctive situations, whether on a nearby machine, within the cloud, or in an on-premises information center. VMs are less convenient due to OS conditions and bigger framework pictures.

Segregation

Holders give process-level confinement but share the have OS bit. This makes them somewhat less secure compared to VMs, which give full confinement by running partitioned OS occasions.

Execution

Since holders don't have the overhead of a complete OS, they are more proficient and offer superior execution. VMs can be slower due to the extra assets required to run numerous OS situations.

Utilize Cases

Holders are perfect for microservices, cloud applications, and CI/CD pipelines due to their speed and adaptability. VMs are way better suited for running numerous OS situations on a single machine, making them valuable for bequest applications and scenarios requiring solid separation.

Conclusion

Holders and virtual machines both have their preferences, and the choice between them depends on the particular utilize case. Holders are best for lightweight, quick, and adaptable applications, whereas VMs are more appropriate for scenarios requiring full OS segregation and security.

Section 4: Docker Architecture

4.1 Introduction to Docker

Docker is an open source standardization containerization platform designed for developer teams and DevOps engineers that has completely revolutionized the software development process in the industries. It has entirely changed the way the product is built, shared or deployed.

Earlier, the development process suffered generally three problems due to different versions of applications and their dependencies-

- a) A code written by a developer on a local machine is written using specific versions of the applications and their dependencies. If another developer joins the project after a long period of time and tries to run that code by again installing the same set of applications and their dependencies in his machine but in the period of time, it may be possible that the newer versions of applications and dependencies have been released. This mismatch between the versions on which the project is built and the latest versions can result in errors in the code.
- b) A code written by a developer team when handled to the tester after year of working on the code may produce the errors because of the version mismatch between the versions available on the systems of developer team and the versions available on the systems of the tester team.
- c) A code written by a developer team and tested by the testing team may result in the errors during the process of deployment because always the latest versions of the applications are installed on the server during deployment.

To counter the above stated problems, the concept of containerization was introduced. Docker enables the developer team to pack applications and their dependencies into standard units called containers. These containers work similar to virtual machines and can run in different environments – development, testing and production without any dependency on the differences in the versions in different environments and this process of containerization has completely eliminated the classic problem of “works on my machine”. The Docker has entirely replaced the concept of virtual machines because the concept of virtual machines has the disadvantage that each virtual machine needs to be run on a separate Operating System which is eliminated in the Docker.

4.2 Key Components of Docker

1. Docker Hub – Docker Hub is a cloud based web application that makes available different predefined Docker images to the developers. The developers can store, manage and share Docker images on the platform. Docker Hub is a repository where developers and companies can pull (download) or push (upload) Docker images. It is a vast library of predefined images of various programming languages such as Node JS and PHP,

databases such as MongoDB, frameworks such as React JS and many more making it easy for developers to easily use the predefined images without any need of creating images of such heavy tools.

2. Docker Desktop – Docker Desktop is a GUI based application that helps developers in building and managing Docker images and containers on their local machines. It can be used via Docker Engine GUI or the Docker Command Line Argument. Docker Engine requires Linux servers to work.

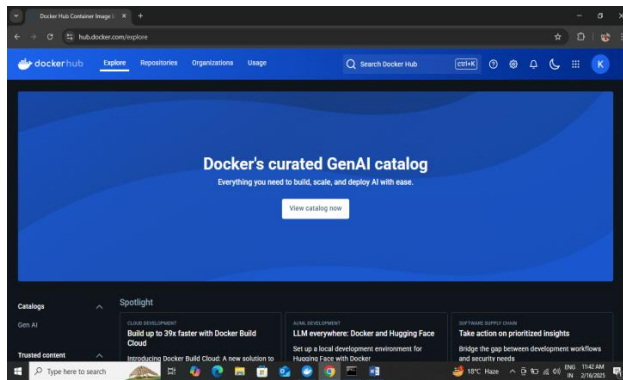


Fig. 4.1 - Docker Hub

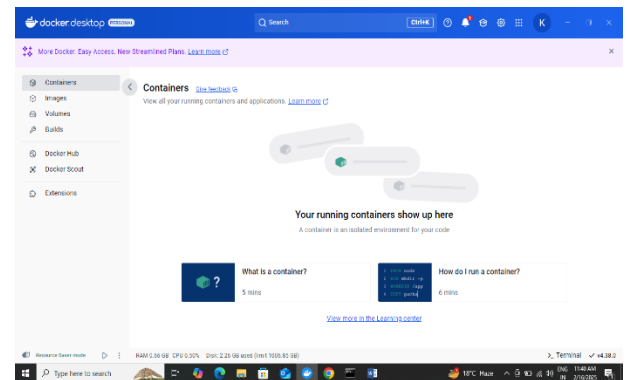


Fig. 4.2 - Docker Desktop

3. Docker Engine – Docker Engine is a system that consists of Docker daemon, Command Line Interface, REST API's, Docker Images, Docker containers and other such related tools.
4. Docker Daemon – Docker Daemon is a background process of Docker Desktop that is responsible for managing various Docker objects – containers, images, networks and volumes.
5. Docker Image – Docker Images are the templates of project or executable packages that contains everything that is needed to run the applications. It consists of source code, application libraries and dependencies, configuration files and environmental variables. Docker images may be predefined or user defined. The predefined images are known as base images or parent images. These parent images are used to build new Docker images.

List all images present in Docker Desktop –

docker images

Pull an image from Docker Hub to the local machine –

docker pull <image-name>

Run the image by making a new container –

docker run <image-name>

Run the image in interactive mode –

docker run -it <image-name>

Delete an image –

docker rmi <image-name>

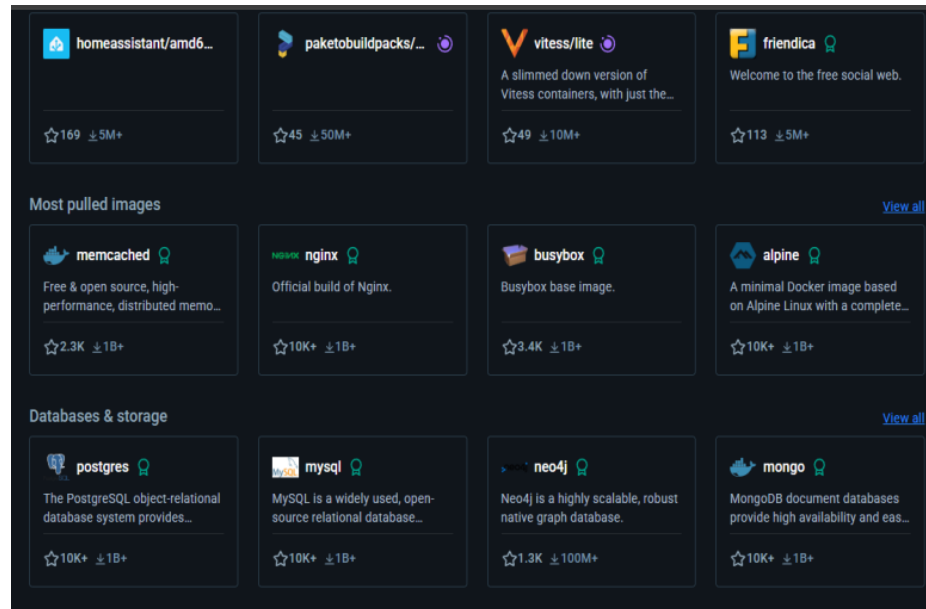


Fig. 4.3 - Docker Images

6. Docker Containers – Docker containers are the runnable instances of the Docker images. Docker container is an environment where the application and its dependencies are run. It ensures that application runs in the same manner across different environments without any version mismatch error.

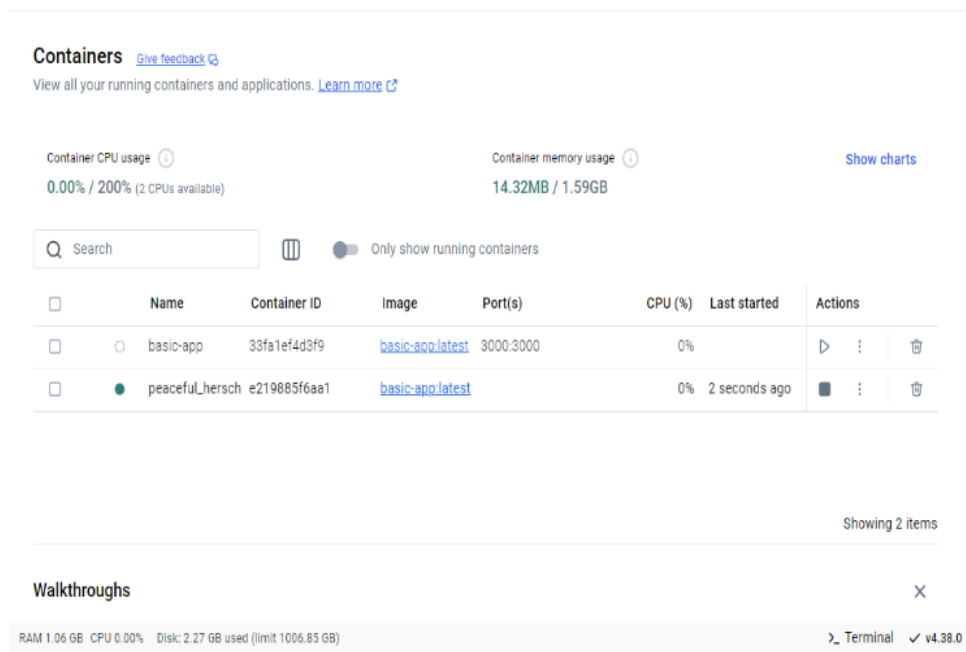


Fig. 4.4 – Docker Containers

List all Docker containers –

`docker ps -a`

List all running containers –

`docker ps`

Start the container –

`docker start <container-name/container-id>`

Stop the container –

`docker stop <container-name/container-id>`

Delete the container –

`docker rm <container-name/container-id>`

Section 5: Advanced Docker Concepts

5.1 Introduction to dockerfile

A Dockerfile is a script file that contains a set of instructions to build a docker image of a particular project. It defines not only the base image and dependencies but also the commands and environmental variables.

The possible entries for the dockerfile are as follows-

FROM <base image>: It specifies the base image of the application. Example – Node.js

WORKDIR: It sets the working directory inside the container

COPY– It copies all project files from the host machine into the container. The first dot specifies the source and the second dot specifies the destination.

RUN – It executes the commands. Example- Installing Node JS dependencies

EXPOSE – It indicates which ports a container's application will be listening on but does not makes the port accessible outside the container.

CMD – It specifies the default command used to run the server. Example- node app.js

ENV – It is used to define the environmental variables. Example- API keys, database credentials

```
\ FROM node:latest
COPY . .
RUN npm install
EXPOSE 3000
CMD ["node","app.js"]
```

Fig. 5.1 - dockerfile

5.2 Introduction to dockerignore file

A dockerignore file is similar to gitignore file that is used to specify files and directories that should not be considered while building the Docker image. This is used to decrease the size of the docker image by excluding unnecessary elements from the Docker image. For a Node based project usually, node modules are written in this file.

```
*.txt
node_modules
```

Fig. 5.2 - dockerignore file

In the above file *.txt denotes all files having the extension .txt .

5.3 Docker Networking

Docker networks are the networks that allows communication between different containers providing isolation between containers. When a container is created, it is assigned to a network where it can interact with other containers.

List all the networks

Command – docker network ls

Create new network

Command – docker network create <network name>

```
C:\Users\Lenovo>docker network ls
NETWORK ID        NAME        DRIVER        SCOPE
dc9277a7577a      bridge      bridge        local
039e4a7ca903      host        host          local
988d0eef0b38      none        null          local

C:\Users\Lenovo>docker network create mongodb
73dce05c3956674c6734741afeea0f295441f05cc780b09cb0a6a5e6e6e3438d

C:\Users\Lenovo>docker network ls
NETWORK ID        NAME        DRIVER        SCOPE
dc9277a7577a      bridge      bridge        local
039e4a7ca903      host        host          local
73dce05c3956      mongodb     bridge        local
988d0eef0b38      none        null          local
```

Fig. 5.3 – Listing and Creating Docker Network

Inspect a Network

Command – docker network inspect <network name>

```
C:\Users\Lenovo>docker network inspect mongodb
[
  {
    "Name": "mongodb",
    "Id": "73dce05c3956674c6734741afeea0f295441f05cc780b09cb0a6a5ebe6e3438d",
    "Created": "2025-03-02T07:13:53.121512603Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "74faee5703ac1a008cfd3f2a9b877c864afec4610f7eed5097247835845eaedc": {
        "Name": "mongo",
        "EndpointID": "7ccf91dfa71ce0ec664f2b4b0aed86066cac8a63f5adb01143b64896616bd0ad",
        "MacAddress": "02:42:ac:12:00:02",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      },
      "ae7a3db3aaff976f939a6c38ceee1654e32e8fabdc9536fc30281119ce70a495": {
        "Name": "mongo-express",
        "EndpointID": "4d5b97dd41713b032822d1471c1757a766ad66eee10d661ce5d102e3d88e4ba7",
        "MacAddress": "02:42:ac:12:00:03",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      }
    },
    "Options": {},
    "Labels": {}
  }
]
```

Fig. 5.4 – Inspecting a Docker Network

The working of docker network is portrayed below using a basic Node JS application where two containers from images mongo and mongo-express are present in a single network mongodb.

```
const express=require('express');
const app=express();
const {MongoClient}=require("mongodb");
app.use(express.urlencoded({extended:true}));
const URL="mongodb://krishna:1234@localhost:27017";
const client=new MongoClient(URL);
app.get("/displayUsers",async(req,res)=>{
  const db=client.db("dockernetwork");
  const data=await db.collection("users").find().toArray();
  res.send(data);
});
app.listen(3000,async()=>{
  await client.connect();
  console.log("Listening to port 3000");
})
```

Fig. 5.5 - Node JS application code

```
C:\Users\Lenovo>docker run -d -p27017:27017 --name mongo --network mongodb -e MONGO_INITDB_ROOT_USERNAME=krishna -e MONGO_INITDB_ROOT_PASSWORD=1234 mongo
74faee5703ac1a008cfd3f2a9b877c864afec4610f7eed5097247835845eaedc
```

Fig. 5.6 – Running a container in Docker Network

Step 1 - Pull the docker image of mongo to setup the database and run the container as seen in the above image.

-d – It signifies that the container is running in detached mode

-p27017:27017 – The default port used for mongodb is 27017. The mongodb runs on the 27017 port of the container and the mapping of this internal port of the container to host machine's port 27017 to access the application from outside of the container is known as **port binding**.

--name mongo – Assigns a name “mongo” to the container

--network mongoddb – Assigns the container to the network mongoddb

-e – Signifies the environmental variables. The username and password are set as environment variables.

mongo – Name of image for which the container is to be made

```
C:\Users\lenovo>docker run -d -p8081:8081 --name mongo-express --network mongoddb -e ME_CONFIG_MONGODB_ADMINUSERNAME=krishna -e ME_CONFIG_MONGODB_ADMINPASSWORD=1234 -e ME_CONFIG_MONGODB_URL="mongodb://krishna:1234@mongo:27017" mongo-express
ae7a3db3aaff976f939a6c38ceee1654e32e8fabdc9536fc30281119ce70a495

C:\Users\lenovo>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
ae7a3db3aaff   mongo-express  "/sbin/tini -- /dock..." 15 minutes ago Up 15 minutes 0.0.0.0:8081->8081/tcp            mongo-express
74faee5703ac   mongo         "docker-entrypoint.s..." 22 minutes ago Up 22 minutes 0.0.0.0:27017->27017/tcp          mongo
```

Fig. 5.7 – Running multiple containers over a Docker Network

Step 2 – Pull the docker image of mongo-express as seen in the above image which provides the user interface to interact with the mongo database setup.

The default port for the mongo-express is 8081 and an additional environment variable of mongoddb URL is mentioned to connect the UI with the database.

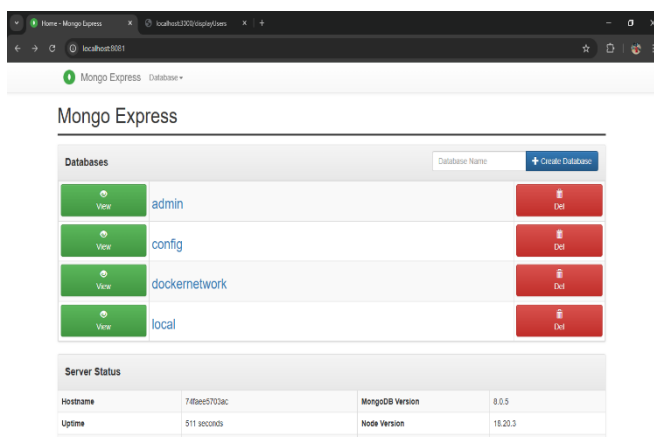


Fig. 5.8 - User Interface of Databases

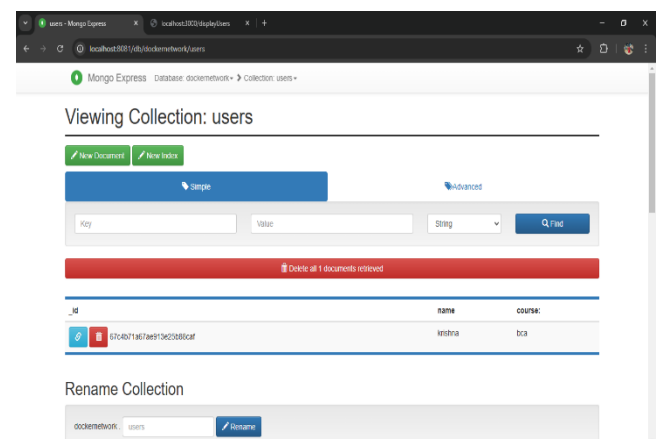


Fig. 5.9 - User collection in the database dockernetwork

Step 3 – A collection is added to the database through the user interface.

Step 4 – Visit the URL as stated in your Node JS code to see the updated changes in the database.

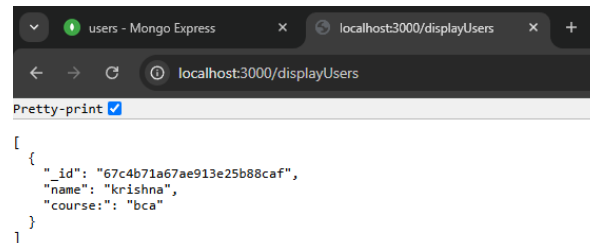


Fig. 5.10 – Updated changes in database

5.4 Docker Compose File

It is seen in the demonstration of docker network that configuring our containers each and every time while running makes the process difficult and complex for the users. To eliminate this complexity, the concept of Docker compose is used. Thus, Docker compose is a tool used to define and run multiple containers by storing all the configurations in a single file with an extension .yaml. Networks are not defined in the yaml file as the network is automatically created by the yaml file.

Run the containers –

`docker compose -f <filename> up -d`

Stop the containers –

`Docker compose -f <filename> down`

In the below demonstration of the Docker compose all the configurations used above (in the Docker network) except the network configuration are listed in the file `compose.yaml`.

```
version: '3.8'

services:

  mongo:

    image: mongo

    ports:

      - 27017:27017

    environment:

      MONGO_INITDB_ROOT_USERNAME: krishna

      MONGO_INITDB_ROOT_PASSWORD: 1234

  mongo-express:

    image: mongo-express

    ports:

      - 8081:8081

    environment:

      ME_CONFIG_MONGODB_ADMINUSERNAME: krishna

      ME_CONFIG_MONGODB_ADMINPASSWORD: 1234

      ME_CONFIG_MONGODB_URL: "mongodb://krishna:1234@mongo:27017/"
```

Fig. 5.11 – Docker Compose File

```
G:\WebDev\BackEnd\DockerNetwork>
G:\WebDev\BackEnd\DockerNetwork>docker compose -f compose.yaml up -d
time="2025-03-02T17:07:43-08:00" level=warning msg="G:\\WebDev\\BackEnd\\DockerNetwork\\compose.yaml: the attribute `version` is obsolete, it will be ignored, please re
move it to avoid potential confusion"
[+] Running 2/2
  mongo-express Pulled                                8.6s
  mongo Pulled                                         8.6s
[+] Running 3/3
  Network dockernetwork_default                       Created                                1.0s
  Container dockernetwork-mongo-1                     Started                               18.3s
  Container dockernetwork-mongo-express-1             Started                               18.3s
```

Fig. 5.12 = Running the containers

```
G:\WebDev\BackEnd\DockerNetwork>docker compose -f compose.yaml down
time="2025-03-02T17:26:30-08:00" level=warning msg="G:\\WebDev\\BackEnd\\DockerNetwork\\compose.yaml: the attribute `version` is obsolete, it will be ignored, please re
move it to avoid potential confusion"
[+] Running 3/3
  Container dockernetwork-mongo-express-1             Removed                                4.5s
  Container dockernetwork-mongo-1                     Removed                                4.1s
  Network dockernetwork_default                       Removed                                1.1s
```

Fig. 5.13 - Stopping the containers

5.5 Docker Volumes

A Docker volume is a storage mechanism that is used to store data permanently. As data stored in the containers is temporary and is lost when the container is stopped or deleted, so the volumes are used to store the data. Multiple containers can share the same volume. Volume is the storage space created on the host machine to store the data of the containerized applications permanently.

List all volumes –

docker volume ls

```
C:\Users\Lenovo>docker volume ls
DRIVER      VOLUME NAME
local       3ae403cd5611d50c5541412a1b5997e7e1f703c617048c3a9e67b953bb14ec1d
local       4a4f123082793d8d801adbbdd6411a2c32fdc6d95eb6e93131bab3194af20451
local       48b74c5c15dbb4f7df3c6718ebad3fc3fc52c852bd3819f76118075bfff53cbee
local       63e99823b40b644dd90bfba51bb16a4277a3af60a88e498a49698d41eab9a787
local       96e92ceb403b9d83a05be085572c2713f4193861bb3b89b558bb52e094634e15
local       52538fbc22f5fb63cffe87edfd21da9a5ea2d6fd067f2fb2bbcc467f6ad85b5f
local       146333b93f4cfae7700f198e7935846076052b858ed7242611c80e338aa9a437
local       c610f9b4632e6cc5542a3b451ba690ebae5b9ead7247cc29c0cef1f3a44231c0
```

Fig. 5.14 – Listing Docker Volumes

```
C:\Users\Lenovo>docker volume create mongo-data
mongo-data
```

Fig. 5.15 – Creating Docker Volume

Create new volume –

docker volume create <volume-name>

Inspect a volume –

docker volume inspect <volume-name>

```
C:\Users\Lenovo>docker volume inspect mongo-data
[
  {
    "CreatedAt": "2025-03-04T03:14:46Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mongo-data/_data",
    "Name": "mongo-data",
    "Options": null,
    "Scope": "local"
  }
]
```

Fig. 5.16 – Inspecting Docker Volume

```
C:\Users\Lenovo>docker volume rm mongo-data
mongo-data
```

Fig. 5.17 – Removing Docker Volume

Remove a volume –

docker volume rm <volume-name>

Remove all unused Docker volumes –

Docker volume prune


```
G:\WebDev\BackEnd\DockerNetwork>docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
ea39792966dde2f03edfb8d82fede1166b624306a5a5a7a1d297718b9d1dac0
0c3e462366c096d2031fc66505a31a93f5f93053a8017fe018a2a16e94342d05
1246ebe4cab1c54406dc4bbd552266c31b4549418e67822fc6533e1a5c900f51
6aae4f65d0756e4a94eeb7e724d0d8086685be8d719aafffc93d7df23ebe5da5
146333b93f4cfae7700f198e7935846076052b858ed7242611c80e338aa9a437
3ae403cd5611d50c5541412a1b5997e7e1f703c617048c3a9e67b953bb14ec1d
4a4f123082793d8d801adbbdd6411a2c32fdc6d95eb6e93131bab3194af20451
63e99823b40b644dd90bfba51bb16a4277a3af60a88e498a49698d41eab9a787
Total reclaimed space: 420.6MB
```

Fig. 5.18 – Prune Docker Volumes

version: '3.8'

services:

mongo:

image: mongo

ports:

- 27017:27017

volumes:

- C:\Users\Lenovo\Desktop\MongoData:/data/db

environment:

MONGO_INITDB_ROOT_USERNAME: krishna

MONGO_INITDB_ROOT_PASSWORD: 1234

mongo-express:

image: mongo-express

ports:

- 8081:8081

environment:

ME_CONFIG_MONGODB_ADMINUSERNAME: krishna

ME_CONFIG_MONGODB_ADMINPASSWORD: 1234

ME_CONFIG_MONGODB_URL:

"mongodb://krishna:1234@mongo:27017/"

Fig. 5.19 – compose.yaml file

Working of Docker Volumes -

Step 1 – Assign the volume to the container using the compose file.

Syntax – local machine storage path: container storage path

Step 2 – Create the database and add some collections in it using mongo-express interface.

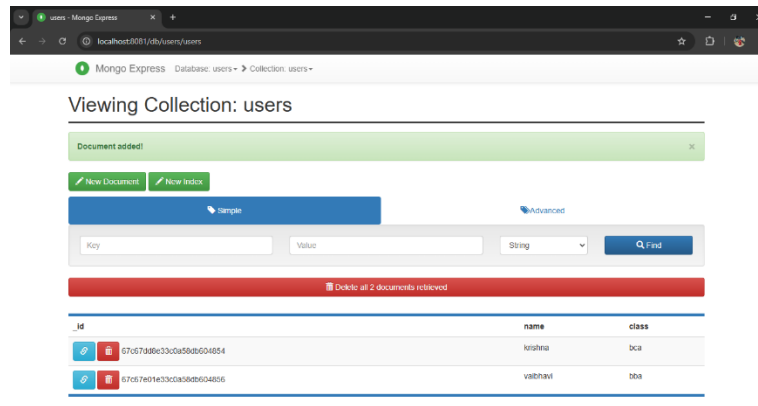


Fig. 5.20 – Collections in users database

Step 3 – Delete the container.

Step 4 – Rerun the container. The databases and collections that have been created before deleting the container would be restored. You can view the persistent storage on the host machine by visiting the specified path.

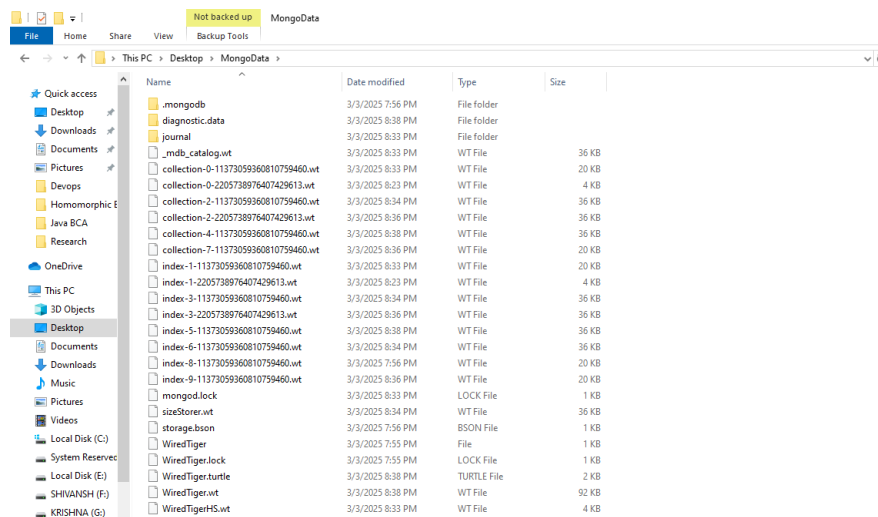
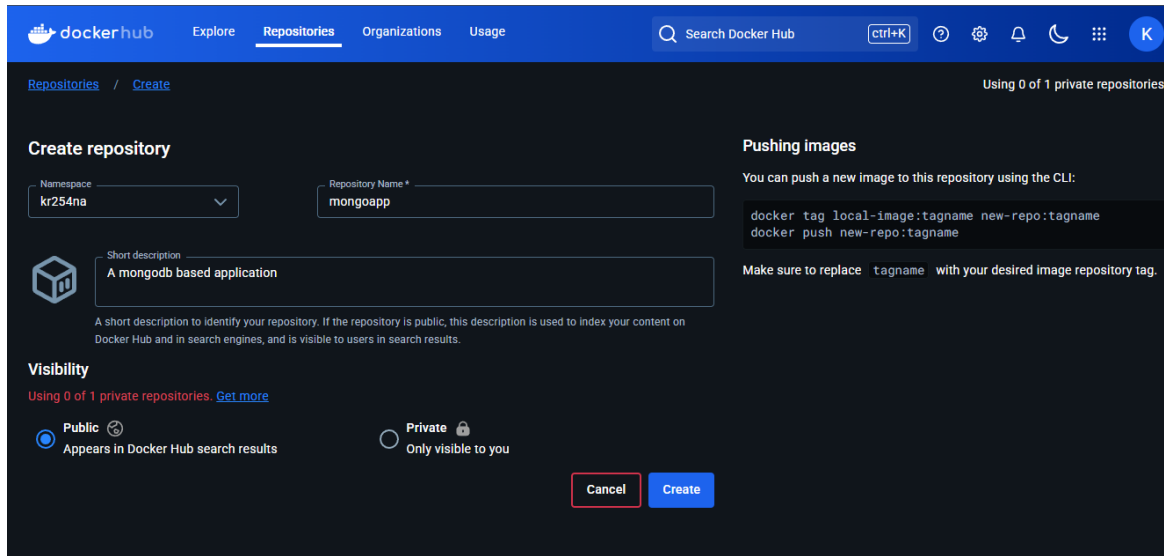


Fig. 5.21 – Persistent Storage of Docker Volumes

Section 6 : Publishing Docker Images

6.1 Publishing Docker Image

Step 1 – Create your public repository on Docker Hub.



The screenshot shows the Docker Hub 'Create repository' page. The 'Namespace' is set to 'kr254na' and the 'Repository Name' is 'mongoapp'. A short description 'A mongodb based application' is entered. The visibility is set to 'Public'. The 'Pushing Images' section shows the CLI commands: `docker tag local-image:tagname new-repo:tagname` and `docker push new-repo:tagname`. The 'Create' button is highlighted.

Fig. 6.1 – Creating public repository on Docker Hub

Step 2 – Build your image with same name as that of repository.

```
G:\WebDev\BackEnd\DockerNetwork>docker build -t kr254na/mongoapp .
[+] Building 6.6s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 198B
=> [internal] load metadata for docker.io/library/node:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/node:latest
=> [internal] load build context
=> => transferring context: 85.06kB
=> CACHED [2/3] RUN mkdir -p Dockernetwork
=> CACHED [3/3] COPY . /Dockernetwork
=> exporting to image
=> => exporting layers
=> => writing image sha256:566f028b20b6363a1df8c568d6d8738cd0f3eadb94677ed65fcf5ae906969464
=> => naming to docker.io/kr254na/mongoapp

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/iptfwp8219jdd5r33guq9ekyv

1 warning found (use docker --debug to expand):
- SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "MONGO_DB_PASSWORD") (line 2)
```

Fig. 6.2 – Building Image

Step 3 – Push the Docker image on your repository.

Command – `docker push <image name>:<version>`

```
G:\WebDev\BackEnd\DockerNetwork>docker push kr254na/mongoapp
Using default tag: latest
The push refers to repository [docker.io/kr254na/mongoapp]
4e87422b26a7: Pushed
9280f4d95161: Pushed
c526f39530b8: Pushing [=====>] 3.584kB
35f49f1f2678: Mounted from library/node
ae3af0a95c24: Preparing
2f40a34baf3e: Waiting
dafb8aed9f7f: Waiting
41d4dc7516bb: Waiting
c0f51bbdc37d: Waiting
91b542912d12: Waiting
```

Fig. 6.3 – Pushing Docker Image

Section 7 : Troubleshooting Commands

7.1 Docker Troubleshooting Commands–

1. `docker logs <container-id>` - Displays the logs of the containers
2. `docker exec -it <container-id> /bin /bash` – Runs the container in the interactive mode where multiple commands can be executed inside the bash
3. `docker exec -it <container-id> / bin /sh` - Runs the container in the interactive mode where multiple commands can be executed inside the shell

Section 8 : Dockerizing our Application

8.1 Time to Dockerize your Application:

Follow the given steps and use the knowledge of explained docker commands to dockerize your own application -

1. Open your project in VS Code
2. Pull / Download the parent / base images
3. Make the docker file, compose file and dockerignore file
4. Build the Docker image of your project
5. Create the container for your image.
6. Run the container
7. Verify the running container.
8. Publish your image on the Docker Hub (if needed)

Section 9 : Kubernetes

9.1 Introduction to Kubernetes

Kubernetes is an orchestration tool used along with the Docker containers to make the process of development smooth and easy. Orchestration is the process of managing, coordinating and automating the Docker containers or microservices. Development process in industries faced major challenges –

1. Server crash during peak load times
2. Management of large number of containers
3. Lack of proper scalability
4. Failure of running containers

Thus, Kubernetes emerged as a solution to these problems.

Criteria	Docker	Kubernetes
Platform Type	Containerization platform	Orchestration platform
Purpose	Consistent environment for application across multiple devices	Management of Docker containers
Self-healing	No self-healing of failed containers	Self-healing of failed containers is a major feature
Scaling	Manual Scaling	Automated Scaling
Use Case	Suitable for small scale projects	Suitable for large scale projects

References –

1. Docker, Inc. "Docker." *linea*.[Junio de 2017]. Disponible en: <https://www.docker.com/what-docker> (2020).
2. El Aouni, Fatiha, et al. "A systematic literature review on Agile, Cloud, and DevOps integration: Challenges, benefits." *Information and Software Technology* (2024): 107569.
3. Faustino, João, et al. "DevOps benefits: A systematic literature review." *Software: Practice and Experience* 52.9 (2022): 1905-1926.
4. Jamsa, Kris. *Cloud computing*. Jones & Bartlett Learning, 2022.
5. Ibrahim, Md Hasan, Mohammed Sayagh, and Ahmed E. Hassan. "A study of how Docker Compose is used to compose multi-component systems." *Empirical Software Engineering* 26 (2021): 1-27.
6. Gokarna, Mayank, and Raju Singh. "DevOps: a historical review and future works." *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*. IEEE, 2021.
7. Acharya, Jigna N., and Anil C. Suthar. "Docker container orchestration management: A review." *International Conference on Intelligent Vision and Computing*. Cham: Springer International Publishing, 2021.
8. Reis, David, et al. "Developing docker and docker-compose specifications: A developers' survey." *Ieee Access* 10 (2021): 2318-2329.
9. Kubernetes, Deploy, and Skills To Deploy Kubernetes Cluster. "Kubernetes." (2021).